

PDF

Jatin Kumar

i have to create this what should be the workflow

Hey Jatin! This looks like a solid, highly practical project for your internship at Griduxa Solutions LLP. Building an end-to-end event management platform for MUNs involves a lot of moving parts—especially since you need to handle real payments and downloadable assets like QR codes and certificates.

PDF + 1

Since your final deadline is **July 25, 2026** (to leave room for submission), a structured, step-by-step development workflow is exactly what you need to avoid getting overwhelmed.

PDF

Here is the ultimate workflow to take you from a blank canvas to a production-ready application.

PDF

Phase 1: Architecture & Database Design (The Foundation)

Before writing UI code, you need to know how your data flows.

1. **Initialize the Project:** Set up a monorepo or separate folders for your `frontend` (React) and `backend` (Node.js/Express).
PDF
2. **Database Schema Design (MongoDB):** Create your Mongoose models based on your requirements.
PDF
 - **User/Participant:** Name, email, password (hashed with bcrypt), portfolio/experience.
PDF
 - **Committee:** Name (e.g., UNSC, DISEC), agenda, total seats, available seats, country matrix.
PDF
 - **Payment:** Transaction ID, status (Pending/Success/Failed), amount.
PDF
 - **Ticket/Attendance:** Unique QR code string, check-in status (Boolean).
PDF
 - **Certificate:** Unique ID, template reference.
PDF

Phase 2: Authentication & Core Event UI (Week 1–2)

Get the user facing portal and basic structure ready.

PDF

1. **Backend Auth:** Build JWT-based login/signup APIs. Create separate roles for **User** and **Admin**.

PDF + 1

2. **Landing Page:** Create a responsive UI showcasing the MUN details, timeline, and available committees.

PDF

3. **Registration Flow:**

- User logs in → fills out the registration form → selects preferred committees/countries from a dropdown populated by your backend.

PDF

Phase 3: Razorpay Payment Gateway (Week 3)

Do not leave payments for the end. This is a critical acceptance criterion.

PDF

1. **Order Creation:** When a user submits their registration, the backend calls the Razorpay API to create an `order_id`.

PDF

2. **Frontend Checkout:** Pass that `order_id` to the frontend and open the Razorpay Checkout modal.

PDF

3. **Payment Verification:**

- **Crucial Step:** Once payment is successful, Razorpay sends back a signature.
- Send this signature to your backend to verify it using your webhook/secret key.
- Only mark the participant's payment status as `Success` after a valid verification.

PDF

Phase 4: Ticketing, Admin Allocation & QR Generation (Week 4)

Now that the user has paid, they need a ticket, and the admin needs control.

PDF

1. **Committee Allocation:** Build the admin dashboard where the organizer can review applicants and officially allot them a country/committee.

PDF

2. **QR Code Generation:** Once allocated, use a library like `qrcode` in Node.js to generate a unique QR code containing the participant's Ticket ID.

PDF

3. **Ticket Download:** Display this ticket on the user dashboard as a downloadable asset.

PDF

4. **Check-in Feature:** Create a simple endpoint `/api/admin/check-in/:ticketId` that changes `Attendance` to `true`. (The admin can literally scan the QR code with their phone to trigger this URL).

PDF +1

Phase 5: PDF Certificate Automation (Week 5)

Reward the participants once the event is "over".

PDF

1. **Validation Rule:** Create a backend check: `If Attendance == true AND Payment == Success` → enable certificate download.

PDF

2. **PDF Generation:** Use libraries like `pdfkit` or `html-pdf` on the backend (or `jspdf` on the frontend) to dynamically inject the participant's name and committee onto a sleek certificate template.

PDF

Phase 6: Deployment & Deliverables (Week 6)

Make it live and polish your submission.

PDF

1. **Deployment:**
 - Deploy the frontend on Render, Vercel, or Netlify.
 - Deploy the backend and MongoDB (Atlas) on Render or Railway.
2. **Environment Variables:** Double-check that your Razorpay live/test keys, JWT secrets, and Mongo URIs are securely hidden in `.env` files.
3. **Documentation:** Write a bulletproof `README.md` including instructions on how to run it locally, an API documentation list, and your schema structure.
4. **Demo Video:** Record a quick 3-5 minute walkthrough showcasing a user registering, paying (in test mode), getting a QR ticket, and the admin approving them.

PDF

PDF

Pro-Tips for Success:

- **Use Razorpay Test Mode:** Don't use real money! Use Razorpay's `rzp_test_...` keys for development.
- **Keep Mentor in the Loop:** Since Mayank Yadav is your mentor, send him a weekly update screenshot or repository link to ensure you're on the right track.
- **Git Commit Often:** A clean commit history looks highly professional when Griduxa evaluates your source code.

PDF

You've got a great tech stack selected. Tackle it feature by feature, and you'll easily cross the finish line before July 25th! Let me know if you want to dive deeper into how to set up any specific API from your list.

PDF + 2

explain the attendance marking feature in detail

Marking attendance efficiently is crucial for a real-world event pilot, as it acts as the gatekeeper for both event entry and certificate issuance.

PDF

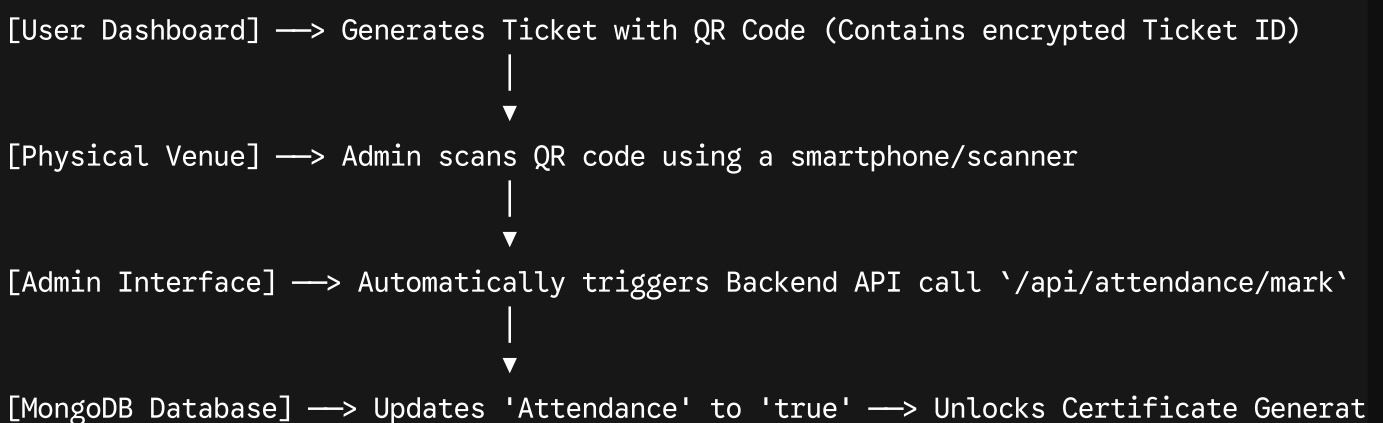
Based on the requirements in [Jatin Kumar.pdf](#), here is a detailed, end-to-end breakdown of how the **QR-based Attendance Marking Feature** should work conceptually, architecturally, and visually.

PDF

The Architectural Flow

The attendance feature bridges the gap between the physical world (the actual event venue) and your digital database. It operates in three main steps:

PDF



Step-by-Step Implementation Detail

1. The Ticket & QR Generation (Pre-Event)

When a participant successfully pays and is assigned a committee by the admin, your system generates their ticket.

PDF

- **The Payload:** Do not store random text in the QR code. Instead, stringify a unique identifier, such as the `Ticket ID` or a combined payload: `{"ticketId": "6671a...", "participantId": "6671b..."}`.
- **The Implementation:** On your Node.js backend, use a library like `qrcode` to convert this string into a Base64 Data URL, embed it onto a clean PDF/HTML layout, and allow the user to download it from their dashboard.

PDF

2. The Scanner Setup (At the Venue Entrance)

You don't need expensive scanner hardware. You can handle this in two ways:

- **The Web-App Approach (Recommended for your React Stack):** Build a dedicated route on your Admin Dashboard (e.g., `/admin/scanner`) that is restricted via your JWT middleware. Use a React library like `react-qr-reader` or `html5-qrcode` to turn the admin's smartphone camera into a live QR scanner.

PDF + 1

- **The Simple Redirect Approach:** Alternatively, the QR code can embed a full URL path: `[https://yourdomain.com/admin/check-in/TOKEN_HERE]` (`https://yourdomain.com/admin/check-in/TOKEN_HERE`). When the admin scans it with their regular phone camera app, it opens the browser, checks their admin login cookie/token, and marks attendance.

3. The Backend API & Database Update

When the scanner successfully reads the QR code payload, it fires an HTTP `POST` or `PUT` request to your backend API.

PDF

- **API Route:** `POST /api/attendance/mark`

PDF

- **Request Body:** `{ "ticketId": "6671a..." }`

- **Backend Logic:**

1. **Authentication Check:** Ensure the incoming request has a valid JWT token showing the user role is `Admin`. Prevent standard participants from hitting this endpoint and marking themselves present.

PDF

2. **Validation:** Check if the `Ticket ID` exists and if the status of the associated payment is `Success`.

PDF

3. **State Mutation:** Update the `Attendance` schema/collection. Change the status from `false` (default) to `true`, and log an accurate `checkedInAt` timestamp.

PDF

4. **Idempotency (Double-Scan Protection):** If the user is already marked present, return a `400 Bad Request` or a message saying *"Participant already checked in at 09:00 AM"*. This prevents chaos at the registration desk if a ticket gets scanned twice.



Impact on the Certificate Workflow

Per the acceptance criteria in [Jatin Kumar.pdf](#), the certificate generation relies entirely on validation rules.

PDF

Once the attendance database field transitions to `true`, it acts as a green light. Your backend code for the certificate route should look structurally like this:

PDF

JavaScript



```
// Example validation check prior to PDF generation
if (participant.paymentStatus === 'Success' && attendance.isPresent === true) {
  // Trigger the PDF certificate generation library
  generateCertificate(participant.name, participant.committee);[cite: 1]
} else {
  return res.status(403).json({ message: "Ineligible for certificate. Please verify"
}
```



Frontend UX Tips for Your Showcase

When building this for your Final Showcase, make sure the Admin Scanning UI includes immediate visual feedback:

PDF

- **Success Flash:** Turn the screen **Green** with a chime sound and display: *"Welcome [Participant Name] - UN Human Rights Council"*.

PDF

- **Error Flash:** Turn the screen **Red** if the payment failed or if the ticket is counterfeit, displaying an explicit error message.

PDF

This guarantees your project functions like a highly polished, enterprise-ready application during your evaluation!

PDF

in simple approach if user scans from his phone he can also verify attendance

That is a highly intuitive UX feature. Allowing participants to scan the code with their own phones to **verify** their registration status or attendance record is an excellent addition.

However, to build this securely, you need to handle the logic differently depending on *who* scans the QR code. You want the **Admin** scan to *change* the attendance status (Marking), but a **Participant** scan to only *view* the status (Verification).

Here is the simple, unified approach to implement this using a single dynamic QR URL.

The Unified QR Code Link Approach

Instead of embedding raw text data into the QR code, embed a full web URL pointing to a dedicated landing page in your React application.

PDF + 1

For example, the QR code payload will be:

```
https://yourmunplatform.com/ticket/verify/TICKET_ID_HERE
```

When anyone scans this QR code using their native phone camera, it will instantly open this webpage in their mobile browser.

Frontend Verification Logic

When that `/ticket/verify/:id` page loads in React, your frontend code should check for the presence of the user's login token (stored in `localStorage` or cookies). This determines what they see on their phone screen:

PDF + 1

Scenario A: The Participant Scans Their Own Ticket

- **What happens:** The app detects they are logged in as a standard user, or matches their user ID with the owner of the ticket.

PDF + 1

- **What they see:** A clean "Live Ticket Passport" screen.
 - **Status Display:** It shows their details and live status:
 -  *Status: Paid & Registered. Present this at the venue entrance.*
 -  *Status: Checked In! Welcome to your Committee.*
- **Security:** They cannot click anything to change their attendance. It is **Read-Only** verification.

Scenario B: An Admin Scans the Ticket

- **What happens:** The app detects that the logged-in user has an `Admin` role.
[PDF + 1](#)
- **What they see:** An operational panel with a prominent action button: "Mark Present" or "Check-In Participant".
- **The Action:** The admin taps the button, which fires the authorized API call to the backend, changing the status to `true`.

Scenario C: A Guest/Logged-Out Person Scans the Ticket

- **What happens:** No active session token is found.
- **What they see:** A basic validity confirmation page showing: "Valid Ticket for MUN 2026. Security Token Verified." No sensitive personal details are leaked.



The Simple Implementation Blueprint

To make this work seamlessly on the backend, you just need two minor adjustments to your existing API design:

[PDF + 1](#)

1. The Verification Route (Public/Participant)

Create a `GET` endpoint that retrieves the public-facing ticket details so the user can verify it.

- **Endpoint:** `GET /api/tickets/verify/:ticketId`
- **Logic:** Finds the ticket, joins it with the participant's name and committee allocation, and returns the current `attendanceStatus` (True/False).

[PDF](#)

2. The Check-In Route (Protected/Admin)

Create a `POST` or `PUT` endpoint that actually updates the database.

[PDF + 1](#)

- **Endpoint:** `POST /api/admin/check-in/:ticketId`
[PDF + 3](#)
- **Logic:** Requires an Admin JWT token. It flips the `Attendance` record field to `true`.

[PDF + 3](#)



Why This Approach Wins For Your Project

- **Zero App Downloads:** Neither the admin nor the participant needs to download a special QR scanning app; they just use their default phone camera.
- **Solves "Self-Verification":** If a participant is sitting in their committee room and wonders, "Did they successfully mark me present?" they can simply scan their own

ticket printed on paper or on their laptop screen to instantly check their live status on their phone.

- **Bulletproof Security:** Even if a participant figures out the URL pattern or copies the link, they can never mark themselves present because the backend endpoint checking for the `Admin` token will block their request.

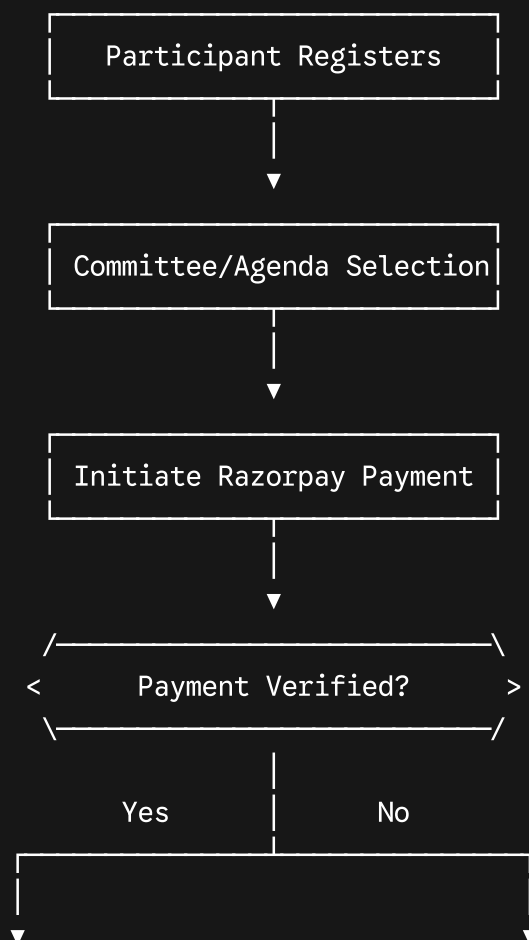
PDF + 1

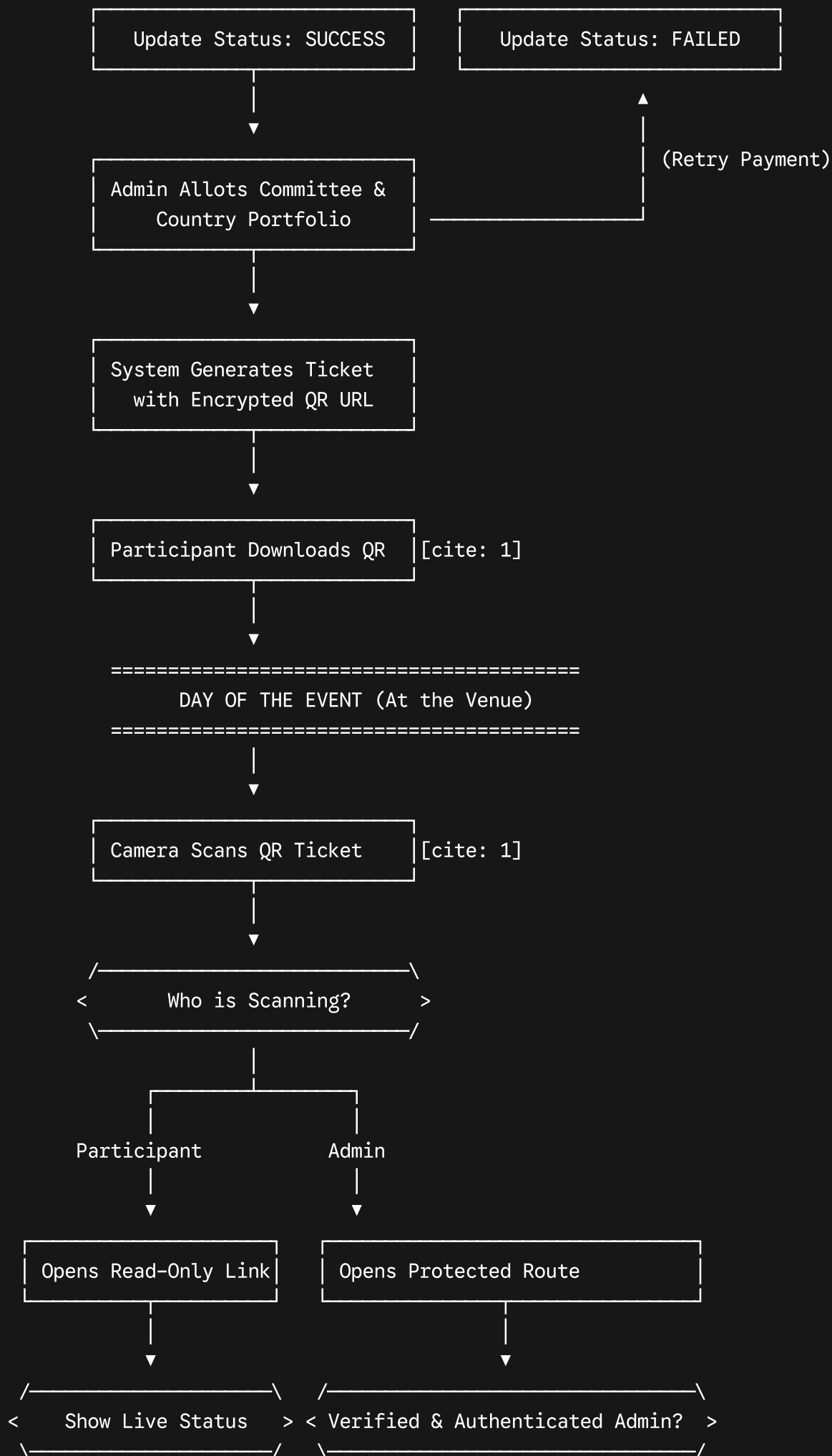
give full flow chart

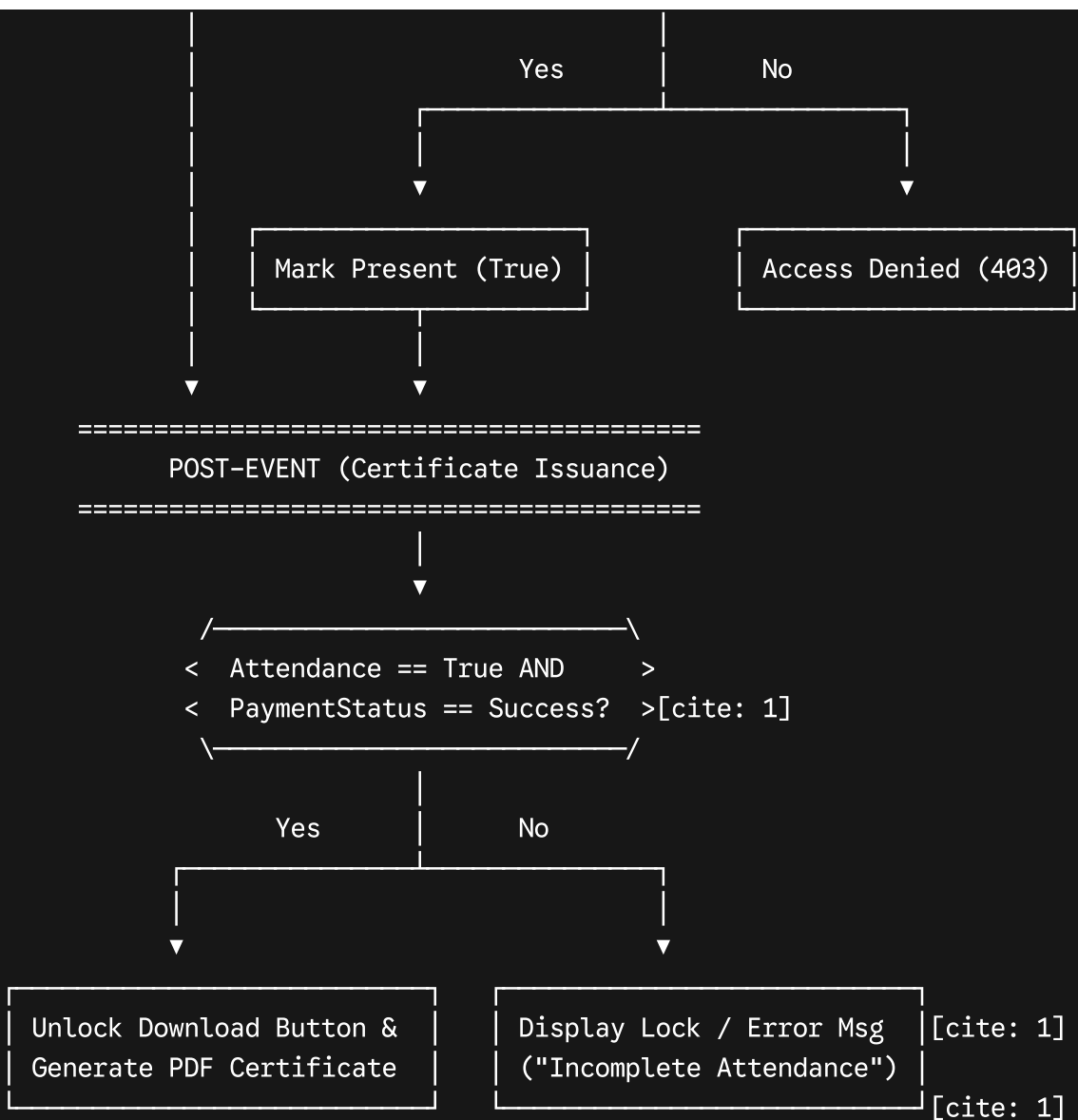
Here is the complete, comprehensive flowchart for your MUN/Youth Parliament platform based on the requirements in **Jatin Kumar.pdf**. It maps out the entire lifecycle of the application, broken down by user roles and operational phases.

PDF + 1

System Architecture & Workflow Diagram







🔍 Breakdown of Key Stages

- **Registration & Gateway Phase (Weeks 1-3):** Captures user data and holds it in a `Pending` state[cite: 1]. Razorpay hooks intercept the response; if the hash signature is verified on the backend, the user state shifts to `Paid` [cite: 1].
- **The Allotment Phase (Week 4):** Admin maps unassigned portfolios from the database schema to the `Participant` profile based on their registration preference[cite: 1].
- **The Live Check-In Split (Week 4):** The QR URL (`/ticket/verify/:id`) acts dynamically[cite: 1]. It serves standard browser requests contextually based on the active JWT user role payload (`User` vs `Admin`)[cite: 1].
- **The Automated Output (Week 5):** The database matches conditional criteria (`attendance: true` and `payment: success`) to pass data fields straight to your PDF generation engine[cite: 1].

any edge cases in this?

Building a real-world platform involving payments, QR code parsing, and automated asset delivery means you are bound to run into edge cases. Anticipating these now will prevent your app from crashing during the final presentation or a live event pilot.

PDF + 1

Here are the critical edge cases you should design for, categorized by feature:

1. Razorpay Payment Edge Cases

- **The "User Closed the Tab" Scenario:**

- *The Issue:* A participant fills out the registration form, opens the Razorpay modal, pays successfully, but closes their browser tab *before* the frontend redirect or verification API can fire. The money leaves their bank account, but your database still says `Pending`.

PDF + 1

- *The Fix:* Do not rely solely on the frontend checkout response. You **must** set up a Razorpay Webhook listener (`payment.captured`) on your backend. When Razorpay processes a payment, its server will ping your webhook directly to update the user's status to `Success` , even if their laptop dies or their browser crashes.

PDF + 1

- **Double Registration Submissions:**

- *The Issue:* A user gets impatient while waiting for the payment gateway to load and double-clicks the "Submit/Pay" button, creating two duplicate registration records or initiating two separate Razorpay orders.
- *The Fix:* Disable the submit button on the frontend the exact moment it is clicked (`loading = true`). On the backend, use unique database indexes (e.g., a compound unique index on `email` + `eventId`) to prevent duplicate entry generation.

2. Committee Allocation Edge Cases

- **The "Overbooking" Problem (Race Conditions):**

- *The Issue:* There is only 1 seat left for the "United States" portfolio in the UNSC committee. Two users open the payment page at the exact same time, pay successfully, and both expect that specific slot.

PDF

- *The Fix:* Use MongoDB transactions or verify seat availability twice—once when creating the Razorpay order, and again *right before* confirming the payment signature on your backend. If the seat filled up during the transaction, flag the user for "Manual Admin Reallocation" instead of letting the application crash or break data integrity.

3. QR Ticket & Attendance Edge Cases

- **The "Spoofed URL" Security Risk:**

- *The Issue:* A participant figures out that your QR code link looks like `.../ticket/verify/6671a` (using sequential or predictable MongoDB ObjectIDs). They alter the ID in the URL to scan themselves in or view another delegate's private information.
- *The Fix:* Do not use raw, predictable data in your public URLs. Encrypt the ticket payload using a short cryptographic token or a UUID, or ensure that your public route strictly hides sensitive data and only exposes a non-sequential unique hash identifier.

- **The Double-Scan (Accidental Check-in):**

- *The Issue:* An admin scans a delegate, but their phone stutters and registers the QR code a second time, triggering the API twice.
- *The Fix:* Your attendance API endpoint must handle idempotency. Check the database state first: `if (ticket.isMarkedPresent) return res.status(200).json({ message: "Already checked in" });`. Avoid continuously overwriting the `checkedInAt` timestamp on accidental scans.

- **Poor Internet Connectivity at the Venue:**

- *The Issue:* The MUN venue has terrible cell reception. The admin tries to scan a QR code, but the React app cannot fetch the API data, leaving a long line of delegates waiting at the door.
- *The Fix:* Build basic local caching or optimization. At the very least, make sure your UI handles API timeouts gracefully by displaying a clear "Network Error: Retrying Check-in..." message rather than freezing up entirely.

4. Certificate Automation Edge Cases

- **The "Name Layout Overflow":**

- *The Issue:* A participant registers with an exceptionally long name (e.g., "H.S.H. Prince Albert Alexandre Louis Pierre Of Monaco"). When your PDF generation engine runs, the text completely overflows past the borders of the certificate template, ruining the design.

- *The Fix:* In your PDF generation code (whether backend or frontend), implement dynamic font scaling. Check the character length of the name string before printing it; if `name.length > 25`, automatically drop the font size down dynamically to fit the boundaries cleanly.
- **The Name Change Request:**
 - *The Issue:* A participant registers using a lowercase typo (e.g., "jatin kumar"), downloads their certificate, and immediately emails support complaining that their official name is wrong.
PDF
 - *The Fix:* Add a "Confirm Name for Certificate" step right before unlocking the download action on the dashboard, allowing users a single opportunity to double-check their spelling before the asset renders permanently.

Rule of Thumb for Your Code:

Always validate your inputs on **both** the frontend and backend. Frontend validation gives a great user experience, but backend validation is what keeps your data safe, accurate, and ready for your final presentation review!

PDF

Are there any specific edge cases here you want to look at from a coding perspective?

PDF

project guide

this guide and techstack overlap?

Yes, absolutely! The new guide provided in [source: 2] overlaps beautifully with the core goals outlined in **Jatin Kumar.pdf**, but it takes your initial workflow and dramatically upgrades it from a "college project" into a highly robust, secure, **production-ready architecture**.

PDF + 1

Think of it as the ultimate upgrade pack for your internship requirements. Here is exactly how they overlap, what has changed, and how you should adapt your development strategy.

PDF + 2



The Alignment Matrix (How They Overlap)

Every single functional requirement given by your mentor, Mayank Yadav, in **Jatin Kumar.pdf** is perfectly accounted for in this production guide:

PDF + 1

Feature Requirement (From PDF)	Production Architecture Equivalent (From Guide)
PDF	PDF
Registration / Portal	Auth Endpoints (<code>/api/auth/register</code>) + Zod Input Validation
PDF	PDF
Committee & Agenda Selection	<code>Committees</code> Collection + Mongo Transactions (Race Condition protection)
PDF	PDF
Razorpay Payment Integration	Backend <code>payment.captured</code> Webhooks (Zero reliance on frontend callbacks)
PDF	PDF
QR Ticket Generation / Check-in	Non-PII Encrypted <code>qrToken</code> + Dedicated Attendance Endpoints
PDF	PDF + 1
Certificate Generation	On-demand PDF creation uploaded to S3/Cloudinary + Cached URL state
PDF	PDF
Admin Dashboard	Middleware-enforced Role-Based Access Control (<code>authorize(["admin"])</code>)
PDF	PDF



Key Upgrades to Your Tech Stack

While your suggested stack in **Jatin Kumar.pdf** was basic JavaScript (`React` , `Node` , `Express` , `MongoDB`), the new blueprint safely introduces enterprise-grade tooling to make your code bulletproof:

PDF + 1

- **TypeScript everywhere:** Using TypeScript across both React and Node/Express catches data type errors during compilation rather than while a live user is paying.

PDF

- **Data Validation (Zod):** Instead of manually checking if a request body has an email or phone number, Zod acts as a strict guardhouse at every endpoint.
PDF
- **File Handling (S3 or Cloudinary):** As explicitly highlighted, **never store PDFs inside MongoDB**. You will generate the certificate, push it to Cloudinary/S3, and store only the resulting string link in your database.
PDF + 1
- **Security Headers & Sanitization:** Adding `helmet`, `express-rate-limit`, and `express-mongo-sanitize` keeps malicious hackers from breaking your API during evaluation.
PDF

Adjusted Step-by-Step Workflow

To build along this production-centric blueprint, you must adjust the design pattern from a basic page-by-page system to a service-oriented architectural flow.

PDF

Phase 1: Folder Separation & Repository Pattern

Instead of writing all your database queries directly inside your endpoint files, split your backend `src/` directory into distinct operational layers:

PDF

1. **Routes:** Direct incoming URL patterns (`/api/payments/verify`) to the correct handler.
PDF
2. **Validators:** Ensure the incoming data parses cleanly through a Zod schema before any execution starts.
PDF
3. **Controllers:** Handle the raw request and response data objects (`req` , `res`).
PDF
4. **Services:** Execute your complex, core business logic rules (e.g., executing `verifyPayment()` or triggering `allocateCommittee()`).
PDF
5. **Repositories:** Directly interact with your database collections using Mongoose methods.
PDF

Phase 2: Atomic Committee Allocation

When executing the allocation step during Week 2 or 4, rewrite the logical check to prevent overbooking. Implement a transaction handler:

PDF + 2

$$\text{Available Capacity} = \text{Capacity} - \text{Filled Seats}$$

If two users apply simultaneously when `Available Capacity = 1`, use an atomic update session (`startSession()`) to safely increment `filledSeats` step-by-step, entirely eliminating race conditions.

PDF

Phase 3: The Asynchronous Enhancement (The "Resume-Worthy" Layer)

Generating PDFs, compiling QR codes, and blasting emails are heavy operations that can easily bottleneck a simple Node.js runtime.

PDF

- Once a webhook successfully processes a payment, do not block the user's browser waiting for a PDF to render.

PDF

- Instead, push that operation onto a background queue management tool like **BullMQ** (powered by Redis). Let the background queue worker securely build the ticket, send the email via SendGrid, and update your DB on its own time.

PDF + 1

By taking the structured milestones outlined in your internship document and implementing them using this production architecture workflow, you will present an application that mirrors real, startup-grade software development.

PDF + 1

